

FREE GUIDE

5 New Relic Alerts Every SaaS Should Have

by Daniel Petrica, New Relic specialist since 2023

Most SaaS teams set up New Relic and stop there. The agents report data. The dashboards look impressive. But nobody configured the alerts. Or worse, they configured too many, and now Slack is a firehose of noise everyone ignores.

These five alerts are the essentials ones I configure first on every production app. They catch real incidents, not phantom spikes. They fire when something is *actually wrong*. And they each take under 10 minutes to set up if you know what threshold to use.

1

Full page load with content validation

What it catches

A blank page that returns HTTP 200.

A REAL INCIDENT

A client's marketing site went silent for four hours. The ping monitor said everything was fine. The status page showed green. The server was returning HTTP 200 with zero errors in the logs.

The page was completely blank. Vercel's edge had a hiccup that caused the HTML payload to be empty. No error code, no stack trace, nothing to alert on. A customer emailed to say "your site is broken."

The fix was a synthetic check that validates the page actually *rendered* something, not just that the server responded.

The alert

A full page load browser test in New Relic Synthetics that loads your homepage and asserts a specific text content exists. If your header text, footer text, or any known stable text is missing, the check fails.

Go to **Synthetics** → **Create monitor** → **Page load performane (Simple browser)**.

Set the check to run every 5 minutes from at least two geographic locations. Alert if it fails twice in a row (avoids transient network blips).

Threshold

One failure = warning to your monitoring channel. Two consecutive failures from different locations = alert to the on-call channel.

Why this matters: A ping monitor telling you the server is reachable is not the same as your customers seeing your page. If you run your frontend behind Cloudflare, Vercel, or any CDN layer, this check catches failures that happen *after* the origin server responds.

Disk space

What it catches

A full disk that prevents log writes, database transactions, file uploads, or SSH access. When disk space hits 100%, your app does not degrade gracefully. It stops working entirely.

The alert

New Relic's infrastructure agent already collects disk metrics. You do not need to install anything extra. Go to **Alerts & AI** → **Alert conditions** → **Create a condition**.

Select **Infrastructure** as the data source, then use this NRQL:

```
SELECT percentage(max(diskUsedPercent), 0) AS 'Disk Used %'  
FROM StorageSample  
WHERE mountPoint = '/'  
FACET entity.name
```

Threshold

Two-stage alert:

Warning at 80%. Send to your monitoring Slack channel. Someone should look within the day. Run `docker system prune -a`, rotate logs, or expand the disk.

Critical at 90%. Page the on-call channel. At 90%, you have hours, not days. The gap between 90% and 100% can be consumed by a single large log file or a runaway process.

Set the condition to evaluate over a 5-minute window to avoid false alarms from temporary spikes (like a Docker image pull).

Why this matters: This is the alert every SaaS forgets. Disk space is boring until it is not. When it is not, your app is down and nobody can SSH in because `/tmp` is full.

APM error rate anomaly

What it catches

A deploy that silently broke an endpoint. A third-party API that started returning errors. A database query that fails under increased load. Anything that causes your application error rate to jump above its normal baseline.

The alert

This uses New Relic's anomaly detection, which learns what "normal" looks like for your app over time. You do not define a fixed threshold (e.g. "alert when error rate exceeds 5%") because that number changes based on traffic patterns. 2% error rate at 3 AM with 10 requests/minute? Probably noise. 2% error rate at peak traffic with 1,000 requests/minute? Something broke.

Go to **Alerts & AI** → **Alert conditions** → **Create a condition**. Select **APM** as the data source. Use **Anomaly** as the condition type with this query:

```
SELECT percentage(count(*), WHERE error IS TRUE) AS 'Error Rate'
FROM Transaction
WHERE appName = 'Your App Name'
```

Threshold

Let New Relic learn your baseline over at least 48 hours before enabling the alert. Configure the sensitivity:

- **Loss of signal** (error rate drops to near-zero). Useful for detecting when the agent stops reporting entirely. Set duration to 15 minutes.
- **Anomaly detection**. Use the moderate sensitivity setting. High sensitivity will fire on minor fluctuations; low sensitivity might miss real incidents. Start moderate and adjust after the first false positive or missed event.

The real work is tuning the sensitivity after your first few alerts. Too aggressive and every minor deploy wakes you up. Too conservative and you find out about incidents from users. That calibration is where experience matters more than documentation.

MySQL connection pool exhaustion

What it catches

Your application running out of database connections. When connection count approaches `max_connections`, new requests queue up, response times spike, and eventually the app returns 5xx errors to every user. This is a classic SaaS outage pattern: connections climb slowly over hours, then everything falls over at once.

The alert

New Relic's MySQL integration collects connection metrics out of the box. Create an alert condition from the infrastructure data source:

```
SELECT latest(threadsConnected) AS 'Connections'  
FROM MysqlSample  
FACET entity.name
```

Threshold

This depends on your configured `max_connections`. Run this on your MySQL server to check:

```
SHOW VARIABLES LIKE 'max_connections';
```

Set the warning threshold at 70% of max connections and critical at 85%. For a typical SaaS app with `max_connections = 151` (the MySQL default), that means:

- **Warning at ~105 connections.** Something is holding connections open. Check for slow queries or connection leaks.
- **Critical at ~128 connections.** You are minutes from exhaustion. Kill long-running queries, restart hung workers, or increase the pool as a temporary measure.

Pair this with a second condition on **slow queries** so you can correlate: was the connection spike caused by one expensive query blocking everyone else, or are connections genuinely leaking?

Why this matters: Connection pool exhaustion is a slow death. By the time users notice, the database has been drowning for 20 minutes. This alert tells you before anyone feels it.

SSL certificate expiry

What it catches

An expired TLS certificate that makes your site unreachable with a browser warning most users will not bypass. This is the 3 AM panic alert: nothing changed in your code, the server is running fine, but your domain shows a security error to every visitor.

The alert

A synthetic monitor that checks your certificate's expiry date. Go to **Synthetics** → **Create monitor** → **Certificate Check**.

Enter your domain and select the check frequency. This is one of the simplest monitors in New Relic. No scripting, no thresholds to tune. It just checks that your certificate is valid and not expiring soon.

Threshold

New Relic handles the thresholding automatically. By default it alerts when the certificate expires in fewer than 30 days. You can tighten this to 14 days if you use automated renewal (Let's Encrypt, ACME) and only want to be alerted if renewal fails. If you manually renew certificates, keep it at 30 days so you have a full billing cycle to act.

Set the check to run once per day. There is no benefit to more frequent checks for certificate expiry. Certificates do not expire mid-day without warning.

Why this matters: This alert takes 30 seconds to configure and prevents the most embarrassing type of outage. A certificate expiry is never a complex technical failure. It is always a "we forgot" failure. Do not be the team that forgot.

What these five cover

Together, these alerts catch:

- Frontend failures invisible to ping monitors (alert 1)
- Infrastructure exhaustion before it becomes an outage (alert 2)
- Code-level errors from deploys and dependencies (alert 3)
- Database connection saturation (alert 4)
- Certificates nobody remembered to renew (alert 5)

These five alerts take a competent developer roughly 3-5 hours to configure properly if they know New Relic. Thresholds need tuning, sensitivity needs calibration, and each one should be tested with a simulated failure to confirm it fires correctly.

And they are still only part of the picture. A production app also needs browser monitoring to catch frontend JavaScript errors, Docker container visibility, web server access and error logs for incident investigation, and alert delivery configured so the right people get paged at the right time.

If you want the complete setup in one week instead of spending a weekend on it:

The Foundation Kit

Full-stack New Relic monitoring configured for your SaaS. APM, host, browser, Docker, synthetics, alerting, MySQL, logs.

€850

One package, one application. Delivered in one week.

[See what is included](#)

Payment plan available. 14-day tuning window included. Foundation Kit documented so any engineer can take over.

Daniel Petrica, New Relic specialist since 2023. 8 years full-stack development. ISO 27001 experience.

danielpetrica.com · [newrelic consulting](#)